Tīmekļa lietotņu izstrāde

Ventspils Augstskola, 2026

2. Projekts

Otrā praktiskā darba gaitā jāizstrādā tīmekļa lietotne ar satura vadības sistēmu (CMS) autorizētiem lietotājiem un publisko lapu.

Saturs

2. Projekts 1
- Datu struktūra 3
- Izstrādes vides sagatavošana un Laravel instalēšana 4
  - Docker 4
  - Wampserver (XAMPP, MAMP, Apache + PHP + MariaDB / MySQL) 6
- Laravel sākumlapas izskats 7
- Git repozitorija izveidošana 8

Pirmā lapa 9
- Kontrolieris 9
- Maršrutēšanas fails 9
- Skati 10
- Vietnes noformēšana 11
- Izmaiņu saglabāšana Github 13

Satura pārvaldība - CRUD 14
- Datubāzu migrācijas 14
- Datubāzes tabula 14
- Modelis 15
- Objektu saraksts 15
- Navigācijas bloks 18
- Jaunu ierakstu pievienošana 19
- Ierakstu rediģēšana 21
- Ierakstu dzēšana 23

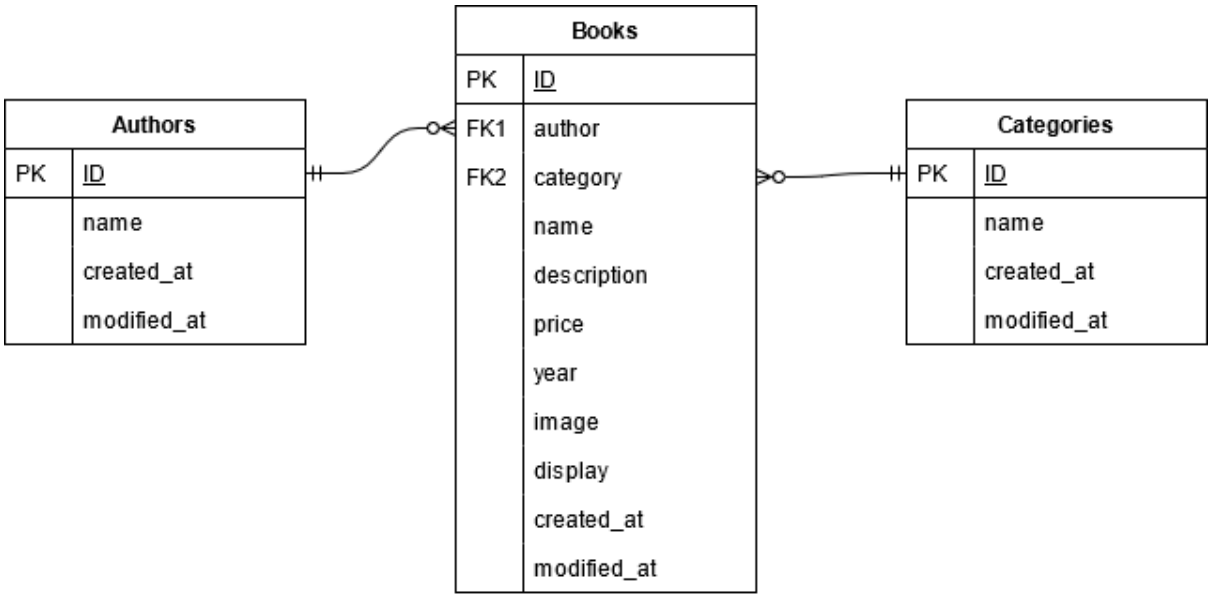
Autentifikācija un autorizācija 26

Galveno datu objektu sadaļa 33

Datubāze un modelis 33
URL adreses 35
CRUD 36
Attēlu augšupielāde 44
Koda pārstrāde 47
Mājas darbs – trešā datu sadaļa 53
Publiskā lapa 54
Stāvoklis, komponentes un lietotnes struktūra 54
HTML + CSS kods publiskajai lapai 55
JSON dati 57
JavaScript būvēšanas rīki 60
Node.js instalēšana, ja izmantojat XAMPP/WampServer 60
Node.js instalēšana, ja izmantojat Docker 61
Node.js instalācijas pārbaude 62
Pakotņu instalēšana 62
Laravel, React un Vite konfigurēšana 63
“Hello, World” React lietotne 65
Pirmās komponentes 70
Testa datu pievienošana 73
Sākumlapas komponentes 74
Izvēlētās grāmatas lapas komponentes 80
Stāvokļa pievienošana 89
Datu ielādēšana no servera 93
Ielādes indikators un tīkla kļūdas 93
HTTP pieprasījumi 95
Izvēlētās grāmatas datu ielāde 103
Saistīto grāmatu ielāde 105
React lietotnes pabeigšana 105
Vietnes papildus noformēšana 106
Cilnes ikona 106
Trešās datu sadaļas risinājums 107
Mājasdarba pamācība 108
Saistīto grāmatu ielādes risinājums 109

Datu struktūra

Sekojošā ilustrācijā attēlota lietotnē izmantotā datu struktūra.



Šajā dokumentā esmu aprakstījis grāmatu kataloga izstrādi – tādu kā tīmekļa veikalu, bet bez iepirkšanās funkcionalitātes.

Centrālais objekts lietotnē būs grāmatas, grāmatām tiks piesaistīti to autori un žanri. Savā projektā varat izmantot citus objektus. Dažas idejas datiem:

- Mūzikas albumi - izpildītāji - žanri
- Auto modeļi - ražotāji - kategorijas
- Filmas - režisori - žanri
- Gleznas - mākslinieki - stili

Izstrādes vides sagatavošana un Laravel instalēšana

Iepriekšējā nodarbībā sagatavojām nepieciešamos rīkus tīmekļa projektu izstrādei- Docker un/vai tīmekļa servera pakotni. Tagad nepieciešams sagatavot izstrādes vidi otrā praktiskā darba projektam. Sekojiet pamācībai, kas piemērota Jūsu izvēlētajiem rīkiem- Docker vai Wampserver servera pakotnei!

Docker

1. Izveidojiet direktoriju savam projektam, piemēram "projekts2"
2. Ar komandrindas lietotni atveriet šo direktoriju
3. Lejuplādējiet Github repozitoriju "2. Projekta sagatave" ar sekojošo komandu (neaizmirstiet punktu beigās!):
   `git clone https://github.com/karlis-i-vea/Project-2-Template.git .`
4. Izdzēsiet direktoriju `.git` – šo Git repozitoriju modificēt nevajadzēs
   Izdzēšot Git repozitorija metadatus, mums paliek tikai 5 projektam nepieciešamie faili, un vēlāk varēsim šeit izveidot jaunu Git repozitoriju.

5. Veiciet Docker konteineru sagatavošanu ar komandu
   `docker compose build`
6. Iedarbiniet Docker konteinerus ar komandu
   `docker compose up -d`
7. Pieslēdzieties Docker konteineram, kurā darbojas PHP, ar komandu
   `docker exec -it project2-laravel bash`
8. Ja atrodaties citā direktorijā, konteinerā aizejiet uz direktoriju `/var/www/src`

9. Izveidojiet Laravel projektu ar sekojošo komandu (neaizmirstiet punktu beigās!):
   `composer create-project laravel/laravel .`
   Laravel dokumentācijā ir minēts cits projekta izveidošanas paņēmiens- ar speciālu Laravel rīku, bet šoreiz izmantosim PHP pakotņu pārvaldnieka Composer dotās iespējas. Ar Composer varat izveidot jebkuru PHP projektu.

10. Konteinerā izpildiet sekojošās komandas:
    `chown -R www-data:www-data storage`
    `chown -R www-data:www-data bootstrap/cache`
    `chmod -R 775 storage`
    `chmod -R 775 bootstrap/cache`
    Šīs Linux komandas tīmekļa serverim piešķir tiesības rakstīt norādītajās direktorijās

11. Laravel konfigurācijas failā `src/.env` uzstādiet datubāzes konfigurācijas parametrus:

```
DB_CONNECTION=mariadb
DB_HOST=project2-database
DB_PORT=3306
DB_DATABASE=project2
DB_USERNAME=project2admin
DB_PASSWORD=project2pass
```

12. Laravel Docker konteinerā izpildiet datubāzes uzstādīšanas komandu `php artisan migrate`

13. Tagad Jums adresē http://localhost/ vajadzētu redzēt Laravel sākumlapu!

Darba beigās izejiet no konteinera ar komandu
`exit`
un pēc tam izslēdziet konteinerus ar komandu
`docker compose down`

Nākošajās reizēs iedarbiniet konteinerus ar komandu
`docker compose up -d`

Wampserver (XAMPP, MAMP, Apache + PHP + MariaDB / MySQL)

1. Wampserver direktorijā `www` izveidojiet direktoriju savam projektam, piemēram, `..\wampserver\www\project2`
2. Atveriet failu `C:\Windows\System32\drivers\etc\hosts` un papildiniet to ar rindām:

```
127.0.0.1 project2
::1 project2
```

3. Atveriet Apache konfigurācijas failu `..\wampserver\bin\apache\apache2.4.X.Y\conf\extra\httpd-vhosts.conf` un papildiniet to ar rindām:

```
<VirtualHost *:80>
    ServerName project2
    DocumentRoot "c:/wampserver/www/project2/public"
    <Directory "c:/wampserver/www/project2/public">
        Options Indexes FollowSymLinks Includes ExecCGI
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

4. Iedarbiniet servera programmas, izmantojot pakotnes kontroles paneli
5. Ar komandrindas lietotni aizejiet uz projekta direktoriju, ko izveidojāt 1. solī
6. Izveidojiet Laravel projektu ar sekojošo komandu (neaizmirstiet punktu beigās!): `composer create-project laravel/laravel .`
7. Laravel konfigurācijas failā `.env` uzstādiet pareizās datubāzes konfigurācijas parametru vērtības norādītajiem parametriem:

```
DB_CONNECTION=mariadb
DB_HOST=127.0.0.1
DB_PORT=3307
DB_DATABASE=project2
DB_USERNAME=root
DB_PASSWORD=
```

8. Komandrindas lietotnē izpildiet datubāzes uzstādīšanas komandu `php artisan migrate`
9. Pārlūkā atveriet adresi http://project2/ un pārbaudiet, ka redzat Laravel sākuma lapu!

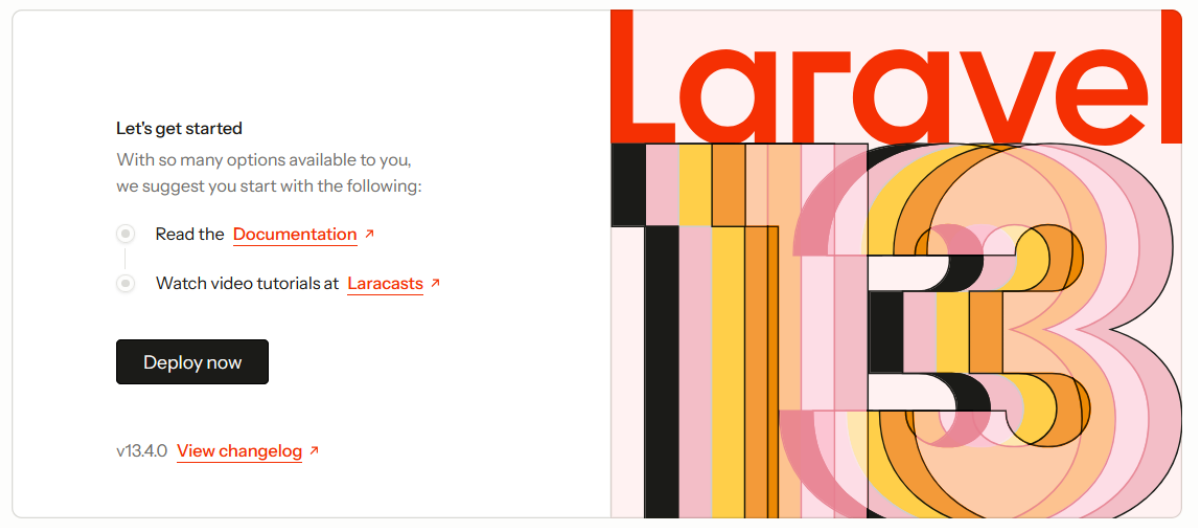
Turpmāk piemēros tiks lietota URL adrese http://localhost/ - Wampserver lietotājiem jāizmanto `hosts` failā definētais domēna vārds – šajā piemērā tas būtu http://project2/!

Darba beigās izslēdziet servera lietotnes, izmantojot pakotnes kontroles paneli!

Nākošajās reizēs iedarbiniet servera lietotnes tāpat, izmantojot kontroles paneli.

Laravel sākumlapas izskats

Veiksmīgas Laravel instalēšanas rezultātā http://localhost/ pārlūkā izskatīsies šādi:



Git repozitorija izveidošana

Izveidosim Git repozitoriju, lai varētu savu projektu droši uzglabāt Github repozitoriju pārvaldniekā!

Pieslēdzieties savam kontam Github, un izveidojiet jaunu projektu!

Github atvērs projekta sākuma lapu, un tajā redzēsiet instrukcijas repozitorija sagatavošanai. Jums jāseko instrukcijām "**...or create a new repository on the command line**", izlaižot pirmo instrukciju, kas izveido jaunu README.md failu!

Atveriet komandrindas lietotni un aizejiet uz savu projekta direktoriju- Wampserver lietotājiem jāiet uz direktoriju, kurā jūs instalējāt Laravel, bet Docker lietotāji var izvēlēties- Git repozitorijam pievienot tikai Laravel direktoriju, vai iekļaut arī Docker konfigurāciju.

Aizejiet uz izvēlēto direktoriju, un izpildiet sekojošās komandas no Github pamācības:

1. `git init`
2. `git add .`
3. `git commit -m "Projekts gatavs darbam"`
4. `git remote add origin` *`GIT-URL`*
5. `git push -u origin master`
   (master vai main, atkarībā no izvēles un iepriekš uzrakstītajām komandām

Ar šīm komandām tiek izveidots lokālais Git repozitorijs, ko piesaista Github serverī izveidotajam projektam. Pēc tam tiek izveidots pirmais iesūtījums, un visu projekta saturu nosūta uz Github.

NB! Pievērsiet uzmanību, lai Git repozitorijā nepievienotu bibliotēku direktorijus, piemēram, `vendor/` un `node_modules/` ! Šie direktoriji satur PHP un JavaScript bibliotēku kodu, kam nav jābūt versiju kontroles sistēmās. Turklāt, tās parasti aizņem daudz vietas. Pašlaik šajā projektā projekta kods ir mazāk par 1MB bet bibliotēku kods- 62MB.

Laravel instalācijā jau ir iekļauts fails `.gitignore` – tajā uzskaitīti faili un direktoriji, ko Git ignorē un neiekļauj iesūtījumos. Laravel projekta .gitignore failā iekļauti bibliotēku, projekta konfigurācijas, izstrādes rīku u.c. faili.

Pirmā lapa

Izveidosim pirmo lapu, ko attēlot vietnes apmeklētājiem. Lai to paveiktu, mums jāizveido kontrolieris ar pieprasījuma metodi, jāpievieno maršrutēšanas ieraksts, jāizveido skata fails un, tā kā šī ir pati pirmā lapa projektā- jāizveido arī visām lapām kopējais HTML šablons.

Kontrolieris

Kontrolieri izveidosim, izmantojot Laravel rīku Artisan. Izpildiet komandu
`php artisan make:controller HomeController`

Pārliecinieties, ka projektā ir izveidots fails
`app/Http/Controllers/HomeController.php` !

Šajā failā importējiet sistēmas klasi `View` un pievienojiet klases metodi `index`:

```
use Illuminate\View\View;

...

    public function index(): View
    {
        return view(
            'home.index',
            [
                'title' => 'Sākumlapa',
            ]
        );
    }
```

Index metode veic tikai vienu darbību- atgriež funkciju `view()` , kas sagatavo HTML dokumentu. `view()` funkcijai tiek padoti divi argumenti- skata faila nosaukums un masīvs ar mainīgajiem, kas būs pieejami skatā. Šo funkciju izmantosiet bieži.

Maršrutēšanas fails

Tagad jāpievieno maršrutēšanas ieraksts. Atveriet failu `routes/web.php` ! Šajā failā jāpievieno maršrutēšanas ieraksts, kas GET pieprasījumus uz adresi http://localhost/ nosūtīs `HomeController` metodei `index`-

```
Route::get('/', [HomeController::class, 'index']);
```

Nodzēsiet failā esošo maršrutēšanas paraugu, lai nedublētu ierakstus pieprasījumam `'/'` . Tā kā šajā ierakstā izmantojam `HomeController` klasi, tad tā ir jāimportē šajā PHP failā- faila augšā pievienojiet importēšanas komandu:

```
use App\Http\Controllers\HomeController;
```

Rezultātā maršrutēšanas failam `web.php` jāizskatās šādi-

```
<?php

use App\Http\Controllers\HomeController;
use Illuminate\Support\Facades\Route;

Route::get('/', [HomeController::class, 'index']);
```

Skati

Vēl atlicis izveidot skatu failus. Direktorijā `resources/views/` izveidojiet failu `layout.blade.php` un tajā iekopējiet HTML dokumenta pamatelementus:

```
<!doctype html>
<html lang="lv">

    <head>
        <meta charset="utf-8">
        <title>2. Projekts - {{ $title }}</title>
        <meta name="description" content="Mans 2. Projekts">
        <meta name="viewport" content="width=device-width, initial-
scale=1">
    </head>

    <body>

        @yield('content')

    </body>

</html>
```

Šis šablons tiks izmantots visu vietnes HTML lapu sagatavošanai, lai mums nevajadzētu katrā lapā atkārtot `<head>` elementu, vietnes navigāciju, u.tml. Pievērsiet uzmanību rindai `@yield('content')` - šī komanda ievietos satura blokus 'content' no skatu failiem šajā šablonā.

Tagad izveidosim pašu skata failu. Direktorijā `resources/views/` izveidojiet direktoriju `home/` , un tajā izveidojiet jaunu failu `index.blade.php` . Šajā failā iekopējiet sākumlapas saturu:

```
@extends('layout')

@section('content')

    <h1>Hello, world!</h1>

@endsection
```

Komanda `@extends('layout')` norāda, ka šis skats izmanto jeb manto skatu no faila "`layout.blade.php`", savukārt komandas `@section()` un `@endsection` apzīmē bloku "`content`", ko iekļaut `layout` skatā.
Šādā veidā var aprakstīt vairākus satura blokus, kā arī veidot garākas skatu mantošanas ķēdes.

Tagad, kad visas sastāvdaļas ir sagatavotas, varam pārlādēt lapu pārlūkprogrammā. Ja viss izdarīts precīzi, redzēsiet savu lapu. Ja ne- Laravel ekrānā attēlos kļūdu paziņojumus.

Kad jaunā sākuma lapa darbojas, varam izdzēst Laravel paraugus- maršrutēšanas ierakstu un skatu `resources/views/welcome.blade.php`

Vietnes noformēšana

Veiciet lapas noformēšanu, izmantojot kādu no CSS satvariem (https://getbootstrap.com/ , https://tailwindcss.com/)

Piemērā es izmantoju Bootstrap satvaru, bet jums nav obligāti jākopē mans lapas noformējums. Vēlāk darba gaitā izmantosim arī Tailwind CSS satvaru.

Layout skatā pievienoju saiti uz Bootstrap CSS failu CDN sistēmā (https://getbootstrap.com/docs/5.3/getting-started/introduction/), kā arī Bootstrap galveni (https://getbootstrap.com/docs/5.3/components/navbar/#text-1) un izmantoju Bootstrap režģi satura un kājenes izvietošanai (https://getbootstrap.com/docs/5.3/layout/grid/).
Rezultātā skata fails tagad izskatās šādi:

```
<!doctype html>

<html lang="lv">

<head>
    <meta charset="utf-8">
    <title>2. Projekts - {{ $title }}</title>
    <meta name="description" content="Tīmekļa Tehnoloģiju 2. praktiskais
darbs">
    <meta name="viewport" content="width=device-width, initial-scale=1">
```

```
    <link

        href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css"
        rel="stylesheet"
        integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
        crossorigin="anonymous"
    >

</head>

<body>

    <nav class="navbar bg-primary mb-3" data-bs-theme="dark">
        <header class="container">
            <a class="navbar-brand" href="#">2. Projekts - {{ $title }}</a>
        </header>
    </nav>


    <main class="container">
        <div class="row">
            <div class="col">

                @yield('content')

            </div>
        </div>
    </main>

    <footer class="text-bg-dark mt-3">
        <div class="container">
            <div class="row py-5">
                <div class="col">
                    K. Immers, 2026
                </div>
            </div>
        </div>
    </footer>

    <script

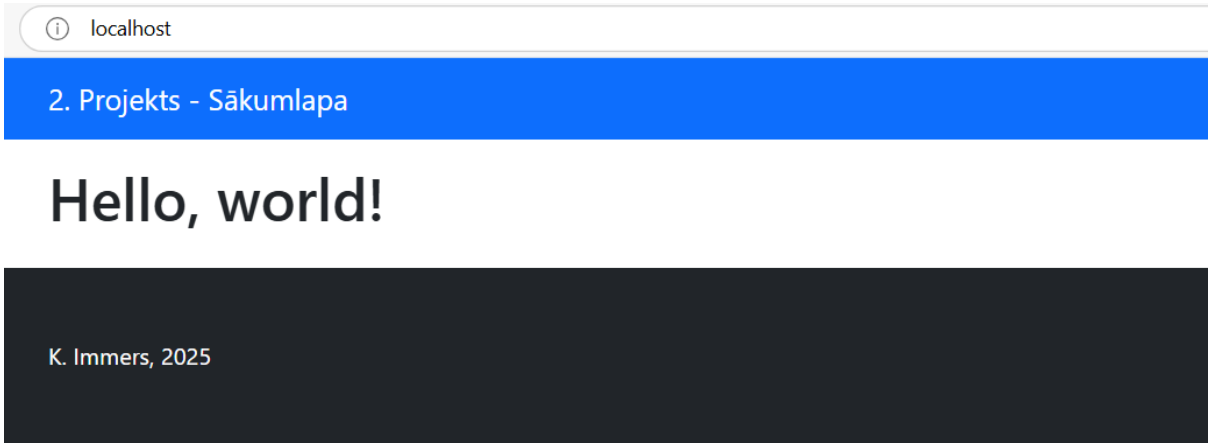
        src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPVlz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
        crossorigin="anonymous"></script>

</body>

</html>
```

Sākuma lapā pagaidām izmaiņas nav jāveic.

Pārlūkā lapa tagad izskatās šādi:



Izmaiņu saglabāšana Github

Pēc pirmās lapas pabeigšanas nosūtiet izmaiņas uz Github!

Tagad izveidosim sistēmas sadaļu, kas pārvaldīs pirmo datu objektu- grāmatu autorus (auto ražotājus, filmu režisorus u.tml.)

Satura pārvaldība - CRUD

Lai tīmekļa lietotnē lietotāji varētu lietot saturu- lasīt rakstus, iepirkties, u.tml- kādam šis saturs ir jāizveido. Būtiska tīmekļa lietotņu sastāvdaļa ir satura pārvaldības sistēmas, un arī šajā projektā nepieciešama iespēja pievienot un rediģēt saturu, ko attēlosim lietotājiem- grāmatas/auto/filmas/albumus u.tml.

Satura pamatoperācijas ir izveidošana, lasīšana, atjaunošana un dzēšana – CRUD (Create, Read, Update, Delete). Mums šajā lietotnē jāizveido trīs CRUD sadaļas- katram objektu veidam sava. Sāksim ar vienkāršāko objektu, šajā piemērā tas būs grāmatu autors.

Datubāzu migrācijas

Laravel sniedz iespēju izmantot datubāzu migrācijas- paņēmienu, kas ievērojami atvieglo darbu ar datubāzēm tīmekļa lietotnēs. Tā vietā, lai datubāzu izmaiņas veiktu manuāli, izmaiņas tiek definētas kodā. Katrai darbībai- tabulu pievienošanai/dzēšanai, kolonnu pievienošanai/mainīšanai/dzēšanai- tiek izveidots jauns fails. Šos failus saglabā versiju kontroles sistēmā tāpat kā citus sistēmas failus, līdz ar to datubāzes konfigurācija tiek piesaistīta konkrētām sistēmas versijām. Laravel kodā ir iekļauts rīks, kas ļauj izpildīt un atcelt sagatavotās migrācijas.

Ar migrāciju palīdzību datubāzes konfigurācija tiek padarīta viegli izsekojama, un tās atvieglo datubāzes *iteratīvu* izstrādi. Migrāciju izmantošana arī ievērojami atvieglo datubāzes sinhronizēšanu vairākās vidēs.

Datubāzes tabula

Datus saglabāsim datubāzē, tādēļ jāizveido datubāzes tabula autoriem. Tabulas veidosim ar migrācijām, tādēļ izmantojam Laravel Artisan rīku, lai izveidotu jaunu migrācijas failu:
`php artisan make:migration create_authors_table`

Laravel parādīs jaunā faila nosaukumu:
`Created Migration: 2022_05_07_070606_create_authors_table`

Atveriet šo failu, tas atrodas direktorijā `database/migrations/`, un papildiniet tā saturu:

```
    public function up(): void

    {
        Schema::create('authors', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('authors');
    }
```

Šī migrācija izveidos tabulu ar 4 kolonnām- `id`, `name`, `created_at` un `updated_at`.

Izpildiet migrāciju ar komandu
`php artisan migrate`

PHPMyAdmin varat pārbaudīt, vai jaunā tabula ir izveidota veiksmīgi. Ja tabulas aprakstā pieļautas kļūdas, varat atcelt pēdējo migrāciju ar komandu
`php artisan migrate:rollback` – tā izpildīs migrācijas metodi `down()`, kas šajā gadījumā izdzēsīs tabulu. Tad varat izlabot migrācijas metodi `up()` un palaist to vēlreiz. Lai labotu vecākas tabulas, vai tādas, kurās jau ir ierakstīti dati, `rollback` gan nevajadzētu izmantot, tās jālabo ar jaunas migrācijas palīdzību.

Modelis

Kad datubāzes tabula gatava, jāizveido modelis, lai varam ērti strādāt ar datiem šajā tabulā. Arī modeli ģenerēsim ar Artisan palīdzību:
`php artisan make:model Author`

Modeļa fails `Author.php` tiek izveidots direktorijā `app/Models/`. Pašlaik šajā failā nekādas izmaiņas nav jāveic – Laravel nodrošina pamatdarbību veikšanu.

Objektu saraksts

Pievienosim savai lietotnei autoru sarakstu – R no CRUD funkcijām! Jāizveido jauna lapa, tādēļ vajadzīgas jau zināmas lietas- kontrolieris, skats, maršrutēšanas ieraksts.

Ar Artisan izveidojam jaunu kontrolieri:
`php artisan make:controller AuthorController`

Kontroliera sākumā importējiet nepieciešamās klases- Autoru modeli un dažas sistēmas klases:

```
use App\Models\Author;
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
```

Pēc tam izveidojiet metodi `list()`:

```
    // display all Authors
    public function list(): View
    {
        $items = Author::orderBy('name', 'asc')->get();

        return view(
            'author.list',
            [
                'title' => 'Autori',
                'items' => $items,
            ]
        );
    }
```

Šīs metodes pirmajā rindā izmantojam **`Author`** modeli, lai ielādētu visus ierakstus no datubāzes tabulas. Ar Laravel iebūvēto metodi **`orderBy`** sakārtojam ierakstus pēc kolonnas **`name`**. Pēc tam atlasītie ieraksti tiek nodoti skatam **`author/list.blade.php`**.

Maršrutēšanas failā **`routes/web.php`** jāimportē jaunā kontroliera klase un jāpievieno maršrutēšanas ieraksts Autoru saraksta lapai:

```
use Illuminate\Support\Facades\Route;
use App\Http\Controllers\HomeController;
use App\Http\Controllers\AuthorController;

Route::get('/', [HomeController::class, 'index']);

Route::get('/authors', [AuthorController::class, 'list']);
```

Izveidojiet jaunu skata failu `resources/views/author/list.blade.php`, un ierakstiet tajā sekojošo kodu:

```
@extends('layout')

@section('content')

    <h1>{{ $title }}</h1>

    @if (count($items) > 0)

        <table class="table table-striped table-hover table-sm">
            <thead class="thead-light">
                <tr>
                    <th>ID</td>
                    <th>Vārds</td>
                    <th> </td>
                </tr>
            </thead>
            <tbody>

            @foreach($items as $author)
            <tr>
                <td>{{ $author->id }}</td>
                <td>{{ $author->name }}</td>
                <td>Labot / Dzēst</td>
            </tr>
            @endforeach

            </tbody>
        </table>

    @else

        <p>Nav atrasts neviens ieraksts</p>

    @endif

@endsection
```

Šajā skatā tiek izveidota tabula, kurā ar `foreach` cikla palīdzību attēlo ierakstus, kas tika ielasīti no datubāzes. Gadījumos, kad nav atrasti ieraksti, ar `if–else` struktūras palīdzību tabulas vietā tiek attēlots paziņojums.

Tabulas stilizēšanai tiek izmantotas Bootstrap CSS klases – ja izmantojat citu CSS bibliotēku, tās jānomaina.

Navigācijas bloks

Sāksim ar navigācijas bloka izveidošanu, lai varētu ērtāk piekļūt vietnes sadaļām. Atveriet galveno HTML šablonu **`/resources/views/layout.blade.php`** un nomainiet visu **`<nav>`** elementa kodu uz sekojošo:

```
<nav class="navbar navbar-expand-md bg-primary mb-3" data-bs-theme="dark">
    <div class="container">
        <span class="navbar-brand mb-0 h1">2. Projekts</span>

        <button class="navbar-toggler" type="button"
            data-bs-toggle="collapse"
            data-bs-target="#navbarNav"
        >
            <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse" id="navbarNav">
            <ul class="navbar-nav">
                <li class="nav-item">
                    <a class="nav-link" href="/">Sākumlapa</a>
                </li>
                <li class="nav-item">
                    <a class="nav-link" href="/authors">Autori</a>
                </li>
            </ul>
        </div>
    </div>
</nav>
```

Ar dzelteno krāsu atzīmētajās vietās ierakstiet savas vietnes nosaukumu, kā arī saiti un nosaukumu objektu sarakstam, ko izveidojāt iepriekšējā nodarbībā!

Galvenes kods ir ņemts no Bootstrap CSS bibliotēkas, varat to papildināt pēc saviem ieskatiem- https://getbootstrap.com/docs/5.3/components/navbar/

Jaunu ierakstu pievienošana

Atveriet objektu skatu `resources/views/author/list.blade.php` un tajā zem tabulas, pēc `@endif` komandas, izveidojiet saiti uz objektu pievienošanas formu. Izmantojot Bootstrap CSS klases, varam šo saiti attēlot kā pogu:

```
<a href="/authors/create" class="btn btn-primary">Izveidot jaunu</a>
```

Maršrutēšanas failā `routes/web.php` pievienojiet divus jaunus ierakstus- lapai, kurā tiks attēlota forma, un pieprasījumam, kas apstrādās formas datus-

```
Route::get('/authors/create', [AuthorController::class, 'create']);
Route::post('/authors/put', [AuthorController::class, 'put']);
```

Pievērsiet uzmanību tam, ka šeit izmantojam gan GET, gan POST pieprasījumu veidus! Abiem pieprasījumiem pat varētu izmantot vienu un to pašu URL adresi, kā tas notiek REST tīmekļa servisos, bet pagaidām nesarežģīsim lietas.

Kontroliera failā `app/Http/Controllers/AuthorController.php` jāpievieno divas jaunas metodes- metode `create()` formas attēlošanai:

```
    // display new Author form
    public function create(): View
    {
        return view(
            'author.form',
            [
                'title' => 'Pievienot autoru'
            ]
        );
    }
```

Un metode `put()` pieprasījuma apstrādei:

```
    // create new Author
    public function put(Request $request): RedirectResponse
    {
        $validatedData = $request->validate([
            'name' => 'required|string|max:255',
        ]);

        $author = new Author();
        $author->name = $validatedData['name'];
        $author->save();

        return redirect('/authors');
    }
```

Šī metode no Laravel saņem mainīgo `$request`, kas satur pieprasījuma datus. Dati tiek validēti, validētos datus ielādē jaunā objektā `Author`, un tad šo objektu saglabā datubāzē. Beigās metode pāradresē lietotāju uz visu objektu skatu.

Izveidojiet jaunu skata failu `resources/views/author/form.blade.php` un iekopējiet tajā sekojošo kodu:

```
@extends('layout')

@section('content')

    <h1>{{ $title }}</h1>

    @if ($errors->any())
        <div class="alert alert-danger">Lūdzu, novērsiet radušās
kļūdas!</div>
    @endif


    <form method="post" action="/authors/put">
        @csrf

        <div class="mb-3">
            <label for="author-name" class="form-label">Autora
vārds</label>

            <input
                type="text"
                class="form-control @error('name') is-invalid @enderror"
                id="author-name"
                name="name">

            @error('name')
                <p class="invalid-feedback">{{ $errors-
>first('name') }}</p>
            @enderror

        </div>

        <button type="submit" class="btn btn-primary">Pievienot</button>

    </form>

@endsection
```

Šajā skata failā atrodas HTML forma, kas sastāv no viena teksta ievadlauka ar parakstu un nosūtīšanas pogas. Formā ir iekļauta arī komanda `@csrf` , kas formā iekļaus slēpto ievadlauku CSRF uzbrukumu novēršanai.

Papildus formas elementiem, skatā ir iekļauti arī validācijas kļūdu paziņojumi.

Tagad vietnē ir iespējams pievienot jaunus `Author` klases objektus!

Ierakstu rediģēšana

Objektu saraksta skatā `resources/views/author/list.blade.php` nomainiet tekstu "Labot" ar rediģēšanas saiti:

```
<a href="/authors/update/{{ $author->id }}" class="btn btn-outline-
primary btn-sm">Labot</a>
```

Pievienojiet divus jaunus maršrutēšanas ierakstus- objekta rediģēšanas formas attēlošanai un pieprasījuma apstrādāšanai:

```
Route::get('/authors/update/{author}', [AuthorController::class, 'update']);
Route::post('/authors/patch/{author}', [AuthorController::class, 'patch']);
```

Pievērsiet uzmanību URL sadaļai `{author}` - šādā veidā Laravel lietotnei tiek nodots identifikators, pēc kā datubāzē atrast ierakstu, ko nepieciešams rediģēt. Tīmekļa lietotnēs kā identifikatorus parasti izmanto ierakstu ID numurus vai tā saucamos *slug* - objekta nosaukumu vai virsrakstu URL adresei derīgā formātā: `item/edit/1234` , `blog/article/glazskunu-rukisi` u.tml.

Šajā Laravel lietotnē mēs izmantosim ierakstu ID numurus. Pateicoties Laravel iebūvētajām funkcijām, mums pašiem nebūs jaraksta kods, kas meklē identifikatoram atbilstošo ierakstu datubāzē, tas notiks automātiski.

Kontrolierī jāpievieno metode, kas attēlos eksistējošu ierakstu rediģēšanas formā:

```
// display Author editing form
public function update(Author $author): View
{
    return view(
        'author.form',
        [
            'title' => 'Rediģēt autoru',
            'author' => $author
        ]
    );
}
```

Kā redzams, šī metode ir gandrīz identiska metodei `create()` - tā lietotājam atgriež formas skatu kopā ar mainīgo `$author`, kurā ir pašreizējās ieraksta vērtības. Norādot metodes argumentu un tā tipu- `Author $author` - mēs norādam Laravel, ka URL adresē esošais identifikators `{author}` jāizmanto, lai datubāzē atrastu pieprasīto `Author` klases objektu. Lai rediģēšanas formā attēlotu pašreizējās objekta vērtības, formas skatam tiek nodots objekts `$author`.

Gan pievienošanas, gan rediģēšanas formām tiek izmantots viens skata fails- `resources/views/author/form.blade.php` , tādēļ papildināsim metodi `create()`, lai skatam nodotu tukšu `Author` objektu:

```
'title' => 'Pievienot autoru',
'author' => new Author()
```

Jāpapildina arī skata fails resources/views/author/form.blade.php.

Nomainiet formas action atribūtu uz sekojošo:

```
action="{{ $author->exists ? '/authors/patch/' . $author->id : '/authors/put' }}"
```

Šī rinda uzstādīs atšķirīgas `action` atribūta vērtības jauna objekta pievienošanas formai un objekta rediģēšanas formai.

Līdzīgi varam nomainīt arī pogas tekstu-

```
{{ $author->exists ? 'Atjaunot autoru' : 'Pievienot autoru' }}
```

Pievienojiet formas teksta ievadlaukam atribūtu value:

```
value="{{ old('name', $author->name) }}"
```

Šī rinda uzstādīs vērtību teksta ievadlaukam- tukšu vērtību jaunam autoram; esošo vērtību autora rediģēšanā; lietotāja ievadīto vērtību validācijas kļūdu gadījumā.

Kontrolierī pievienojiet metodi rediģēšanas pieprasījuma apstrādei:

```
// update existing Author data
public function patch(Author $author, Request $request): RedirectResponse
{
    $validatedData = $request->validate([
        'name' => 'required|string|max:255',
    ]);

    $author->name = $validatedData['name'];
    $author->save();

    return redirect('/authors');
}
```

Arī šī metode ir gandrīz identiska jauno ierakstu saglabāšanas metodei `put()`. Vienīgā atšķirība ir tāda, ka rediģēšanas metodē netiek izveidots jauns `Author` objekts, tā vietā metode saņem `$author` objektu, ko Laravel ir ielādējis no datubāzes, izmantojot ID no pieprasījuma URL adreses. Tāpat kā jauno ierakstu metodē, arī rediģēšanas metodē tiek

veikta klienta atsūtīto datu validācija, pēc tam datus ievieto `$author` objektā, un objektu saglabā datubāzē. Beigās lietotājs tiek pāradresēts uz autoru sarakstu.

Tagad vietnē ir iespējams rediģēt iepriekš izveidotos objektus!

Ierakstu dzēšana

Atlikusi vēl viena darbība- ierakstu dzēšana. Saraksta lapā pievienosim formu, ar ko veikt HTTP POST pieprasījumu uz serveri, un kontroliera klasei pievienosim metodi, kas veiks ierakstu dzēšanu.

Pievienojiet maršrutēšanas ierakstu dzēšanas pieprasījumam:

```
Route::post('/authors/delete/{author}', [AuthorController::class, 'delete']);
```

Objektu saraksta skatā `resources/views/author/list.blade.php` nomainiet tekstu "Dzēst" ar sekojošo formu:

```
<form action="/authors/delete/{{ $author->id }}" method="post" class="deletion-form
d-inline">
    @csrf
    <button type="submit" class="btn btn-outline-danger btn-sm">Dzēst</button>
</form>
```

Kontrolierī pievienojiet dzēšanas metodi:

```
public function delete(Author $author): RedirectResponse
{
    // šeit derētu pārbaude, kas neļauj dzēst autoru, ja tas piesaistīts
eksistējošām grāmatām
    $author->delete();
    return redirect('/authors');
}
```

Kā redzams, šī metode tikai izdzēš ierakstu un pāradresē lietotāju.

Lai novērstu iespēju izdzēst ierakstus nejauši, pievienosim dialoglogu, kas lietotājiem lūgs apstiprināt ieraksta dzēšanu.

HTML šablona failā `/resources/views/layout.blade.php` pirms aizverošā `</body>` taga pievienojiet saiti uz JavaScript failu:

```
<script src="/js/admin.js"></script>
```

Izveidojiet šo JavaScript failu `public/js/` direktorijā, un iekopējiet tajā sekojošo kodu:

```
"use strict";

function setupDeleteForms() {

    let deleteForms = document.querySelectorAll('form.deletion-form');


    for (let form of deleteForms) {
        form.addEventListener('submit', function (event) {
            event.preventDefault();

            if (window.confirm('Vai tiešām vēlaties dzēst šo ierakstu?')) {
                form.submit();
            } else {
                return false;
            }
        });
    }
}

document.addEventListener("DOMContentLoaded", function () {
    setupDeleteForms();
});
```

Šis JavaScript kods veic šādas darbības:

Tiek definēta funkcija `setupDeleteForms()`:

1. Funkcijā, izmantojot JavaScript funkciju `querySelectorAll()`, tiek sameklēti visi `<form>` elementi ar klasi "`deletion-form`"
2. `for` ciklā katrai formai tiek pievienots *notikumu klausītājs* (event listener), kas reaģē uz formas nosūtīšanu ("`submit`" notikums)
3. Kad lietotājs noklikšķina uz sarkanās dzēšanas pogas, notikuma klausītājs uztver šo notikumu, un izsauc tam piesaistīto anonīmo funkciju
4. Anonīmā funkcija aptur formas nosūtīšanu ar funkciju `preventDefault()`
5. Tad tiek izsaukts dialoglogs `confirm`, kas liek lietotājam izvēlēties atbildi "Jā" vai "Nē"
6. `if` struktūrā, atkarībā no lietotāja atbildes, tiek atjaunota formas nosūtīšana, vai arī izbeigta funkcijas darbība.

Rindā `document.addEventListener("DOMContentLoaded")` HTML dokumentam tiek izveidots notikumu klausītājs, kas reaģē uz notikumu "`DOMContentLoaded`" – pārlūkprogramma signalizē par šo notikumu brīdī, kad viss dokumenta HTML kods ir ielādēts no tīmekļa.
Notikuma klausītājs tad izsauc funkciju `setupDeleteForms()` .

Īsumā – tad, kad lietotājs mēģina nosūtīt dzēšanas formu, formas nosūtīšana tiek apturēta, un ar dialogloga palīdzību lietotājam lūdz apstiprināt formas nosūtīšanu. Ja lietotāja atbilde ir noraidoša, funkcijas izpilde tiek pārtraukta. Ja lietotāja atbilde ir apstiprinoša, tiek veikts HTTP pieprasījums uz serveri, kur ieraksts tiek izdzēsts.

Kad esat notestējuši visus pieprasījumus un izlabojuši kļūdas, ja tādas ieviesušās, nosūtiet izmaiņas uz Github!

Autentifikācija un autorizācija

Lai novērstu iespēju neautorizētiem lietotājiem veikt izmaiņas savas lietotnes saturā, pievienosim autorizācijas un autentifikācijas funkcionalitāti. Laravel piedāvā dažādas iespējas autorizācijas un autentifikācijas realizēšanai, un ļoti vēlams ar tām iepazīties, ja vēlaties izstrādāt publiski pieejamu lietotni- https://laravel.com/docs/13.x/authentication

Šajā darbā izveidosim minimālu autentifikācijas mehānismu, vienlaicīgi ievērojot labās prakses principus.

Minimālai autentifikācijai/autorizācijai nepieciešami trīs HTTP pieprasījumi- HTML formas attēlošana, lietotāja pieslēgšanās mēģinājuma apstrāde, un lietotāja darba sesijas izbeigšana. Ar Laravel Artisan rīku izveidojiet jaunu kontrolieri, kas apstrādās šos pieprasījumus:

```
php artisan make:controller AuthController
```

Maršrutēšanas failā **routes/web.php** pievienojiet maršrutēšanas ierakstus:

```php
use App\Http\Controllers\AuthController;

...

// Auth routes
Route::get('/login', [AuthController::class, 'login'])->name('login');
Route::post('/auth', [AuthController::class, 'authenticate']);
Route::get('/logout', [AuthController::class, 'logout']);
```

Izveidojiet jaunu skata failu **resources/views/auth/login.blade.php** un iekopējiet tajā HTML formas kodu:

```
@extends('layout')

@section('content')

    <h1>{{ $title }}</h1>

    <hr>

    @if ($errors->any())
        <div class="alert alert-danger">
            Neizdevās pieslēgties. Lūdzu, mēģiniet vēlreiz!
        </div>
    @endif
```

```
    <form method="post" action="/auth">
        @csrf

        <div class="mb-3">
            <label for="login-name" class="form-label">Lietotāja vārds</label>
            <input
                type="text"
                id="login-name"
                name="name"
                class="form-control"
                autocomplete="off"
                autofocus
            >
        </div>

        <div class="mb-3">
            <label for="login-password" class="form-label">Parole</label>
            <input
                type="password"
                id="login-password"
                name="password"
                class="form-control"
            >
        </div>

        <div class="mb-3">
            <button type="submit" class="btn btn-primary">Pieslēgties</button>
        </div>

    </form>

@endsection
```

Šī ir vienkārša HTML forma ar diviem ievadlaukiem- lietotājvārds un parole. Lietotājvārda ievadlaukam ir izslēgta *autocomplete* funkcionalitāte, un tajā automātiski tiek novietots kursors- neliels, bet pamanāms lietotāju pieredzes uzlabojums.

Autorizācijas kontrolierī `app/Http/Controllers/AuthController.php` pievienojiet nepieciešamos importus un metodi `login()`, kas lietotājam attēlos pieslēgšanās formu, un metodi `authenticate()`, kas apstrādās autentifikācijas pieprasījumu:

```php
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
use Illuminate\Support\Facades\Auth;

...

// display login form
public function login(): View
{
    return view(
        'auth.login',
        [
            'title' => 'Pieslēgties'
        ]
    );
}

// authenticate user
public function authenticate(Request $request): RedirectResponse
{
    $credentials = $request->only('name', 'password');

    if (Auth::attempt($credentials)) {

        $request->session()->regenerate();

        // Pāradresācijas URL vēlāk nomainīsim uz /books
        return redirect('/authors');
    }

    return back()->withErrors([
        'name' => 'Autentifikācija neveiksmīga',
    ]);
}
```

Autentifikācijas metodē no HTTP pieprasījuma objekta tiek paņemti mainīgie `name` un `password`, un tad Laravel iebūvētā funkcija `Auth::attempt()` veic visus autentifikācijas procesus- ievaddatu filtrēšanu, paroles *jaukšanu* (hashing), un lietotāja datu pārbaudi datubāzē – jums pašiem nekas no tā nav jādara.
Veiksmīgas autentifikācijas gadījumā tiek pārģenerēts lietotāja sesijas identifikators, un lietotāju pāradresē uz galveno satura sadaļu, šajā piemērā- `/books`.

Lai varētu pieslēgties sistēmā, ir nepieciešams lietotāja ieraksts datubāzē. Ir vairāki veidi, kā tos izveidot, bet tā kā mums nepieciešams tikai viens lietotāja konts sev pašiem, manuāli ierakstīsim to datubāzē.

Paroles datubāzē tiek saglabātas šifrētā jeb *sajauktā* (hashed) veidā, tādēļ izmantosim Laravel, lai sagatavotu paroles kodu. `login()` metodē ierakstiet šādu rindu pirms `return()` komandas:

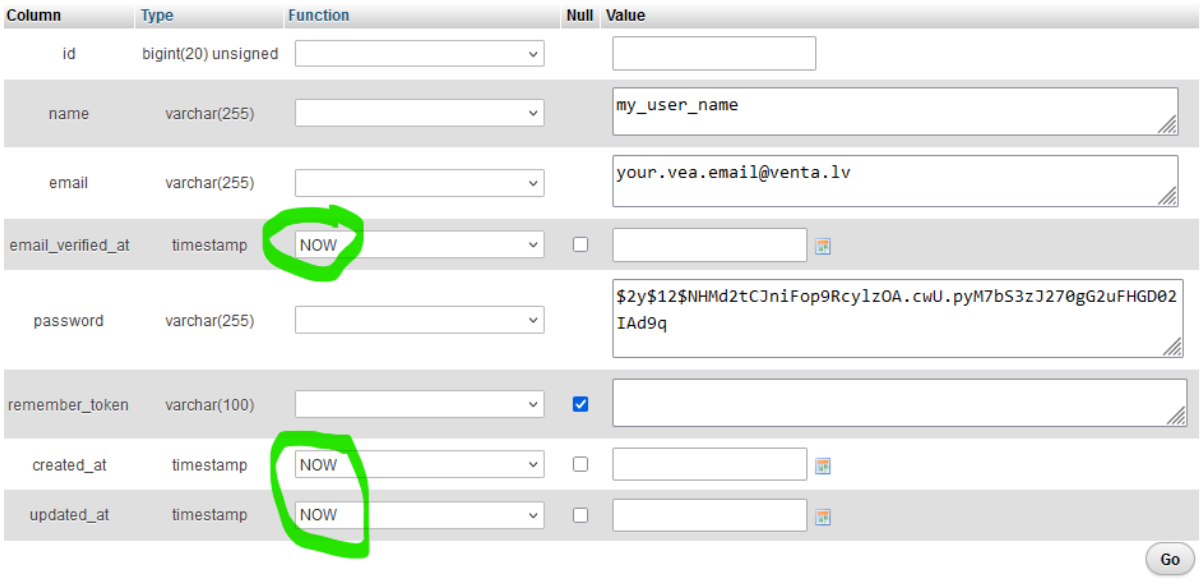
```
echo 'mans_lietotāja_vārds : <input value="'
. \Illuminate\Support\Facades\Hash::make('mana_parole_123')
. '">';
exit();
```

(Protams, lietotāja vārdu un paroli norādiet tādu, kādu vēlaties!)

Pārlūkā redzēsiet šādu lapas saturu-

mans_lietotāja_vārds : $2y$10$YsP.bjEa.D1.wj

Nokopējiet ievadlaukā redzamo simbolu virkni un, izmantojot savu iecienīto datubāzu pārvaldības rīku, projekta datubāzes tabulā `users` pievienojiet jaunu ierakstu ar atbilstošajām vērtībām:
(Docker lietotājiem pieejams PhpMyAdmin- http://localhost:8080/)



Kad lietotājs izveidots, izdzēsiet paroles sagatavošanas rindu no `login()` metodes un pamēģiniet pieslēgties savā lietotnē!

⚠ Protams, reālajā dzīvē šādā veidā neviens lietotājus neveido, bet tagad mēs esam veikuši visas tās darbības, ko mūsu vietā parasti dara satvari, un zinām, kas notiek "zem motora pārsega"

Kad autentifikācija/autorizācija darbojas, papildināsim lietotnes navigāciju- saites uz satura sadaļām rādīsim tikai autorizētiem lietotājiem, bet neautorizētajiem- tikai saiti uz pieslēgšanās formu. Pievienosim arī iespēju autorizētajiem lietotājiem beigt darbu sistēmā. Atveriet skata failu `resources/views/layout.blade.php` un nomainiet navigācijas sadaļu:

```
...
@if(Auth::check())

    <li class="nav-item">
        <a class="nav-link" href="/authors">Autori</a>
    </li>

    <li class="nav-item">
        <a class="nav-link" href="/logout">Beigt darbu</a>
    </li>

@else

    <li class="nav-item">
        <a class="nav-link" href="/login">Autentificēties</a>
    </li>

@endif
...
```

Komanda `@if(Auth::check())` pārbauda, vai lietotājs ir autorizējies, un sagatavo atbilstošās navigācijas saites.

Visbeidzot, autorizācijas kontrolierī pievienojiet metodi `logout()` , kas pārtrauks autorizēto lietotāju sesijas:

```
// end user session
public function logout(Request $request): RedirectResponse
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect('/');
}
```

Vēl atlicis aizliegt neautorizētiem lietotājiem piekļūt mūsu satura vadības sistēmai. Lai to izdarītu, papildiniet Autoru kontrolieri ar sekojošo interfeisu un statisko metodi:

```
...
use Illuminate\Routing\Controllers\HasMiddleware;

class AuthorController extends Controller implements HasMiddleware
{
    /**
     * Get the middleware that should be assigned to the controller.
     */
    public static function middleware(): array
    {
        return [
            'auth',
        ];
    }
...
```

Tagad Autoru kontrolieris izmantos Laravel iebūvēto funkcionalitāti, lai pārbaudītu lietotāja autentifikācijas statusu katrā pieprasījumā. Tikai autorizētie lietotāji varēs piekļūt Autoru sadaļai.

Pārliecinieties, ka pieslēgšanās/atslēgšanās darbojas, pārbaudiet, vai satura vadības lapas ir pieejamas tikai autorizētam lietotājam, un tad saglabājiet izmaiņas Git!

Galveno datu objektu sadaļa

Iepriekšējā nodarbībā izveidojām papilddatu pārvaldes sadaļu (šajā piemērā- grāmatu autori). Tagad izveidosim centrālo datu objektu- šajā piemērā grāmatu- sadaļu.

Datubāze un modelis

Sāksim ar datubāzes sagatavošanu. Šajā tabulā būs vairāk kolonnu (sk. datu struktūru) - teksta un skaitļu kolonnas, kolonnas citu objektu piesaistīšanai, u.tml.

Izmantojot Laravel Artisan rīku, izveidosim migrāciju, modeli un kontrolieri ar vienu komandu:
`php artisan make:model -mc Book`

Kaut arī šī ir modeļu veidošanas komanda, ar slēdzi `-mc` tai tiek uzdots izveidot arī migrāciju un kontrolieri modelim, kas tiek izveidots. Strādājot ar tīmekļa satvariem ir vērts iepazīties ar to piedāvātajām iespējām, kas ļauj strādāt efektīvāk.

Atveriet tikko izveidoto migrācijas failu, un definējiet tajā jaunās datubāzes tabulas kolonnas:

```php
public function up(): void
{
    Schema::create('books', function (Blueprint $table) {
        $table->id();
        $table->foreignId('author_id');
        $table->string('name', 256);
        $table->text('description')->nullable();
        $table->decimal('price', 8, 2)->nullable();
        $table->integer('year');
        $table->string('image', 256)->nullable();
        $table->boolean('display');
        $table->timestamps();
    });
}
```

Pievērsiet uzmanību deklarācijai `$table->foreignId('author_id');` - tā norāda, ka datubāzes tabulā `books` jāizveido kolonna `author_id`, kurā tiks norādīts konkrētajam grāmatas ierakstam piesaistītā autora ieraksta ID.

Kolonnas `author_id` nosaukumu pielāgojiet savai lietotnei!

Lai pašā datubāzē izveidotu *ārējo atslēgu* (foreign key), izveidosim vēl vienu migrāciju- `php artisan make:migration alter_table_books`

Tajā ierakstiet šādu kodu:

```php
public function up(): void
{
    Schema::table('books', function (Blueprint $table) {
        $table->foreign('author_id')->references('id')->on('authors');
    });
}

public function down(): void
{
    Schema::table('books', function (Blueprint $table) {
        $table->dropForeign('books_author_id_foreign');
    });
}
```

Šī migrācija izveidos ārējo atslēgu pašā datubāzē. Šo procesu esmu atdalījis atsevišķā migrācijā, jo reizēm, atkārtoti izveidojot projekta datubāzi no jauna, Laravel nemāk izveidot gan tabulu, gan tās atslēgas vienā operācijā.
Atcerieties nomainīt nosaukumus `books`, `authors`, `author_id` un `books_author_id_foreign` uz tādiem, kas atbilst Jūsu datubāzei!

Izpildiet migrācijas un pārliecinieties, ka tabula izveidota pareizi! Komanda migrāciju izpildīšanai:
`php artisan migrate`

Tā kā grāmatu un autoru klases ir saistītas, jāsagatavo datu relācijas – attiecība/saikne starp šo divu klašu objektiem. Atveriet abus modeļus direktorijā `app/Models` - `Author.php` un `Book.php`.

Autoru modelī jāpievieno atsauce uz grāmatu klasi:

```
use Illuminate\Database\Eloquent\Relations\HasMany;
...
public function books(): HasMany
{
    return $this->hasMany(Book::class);
}
```

Grāmatu modelī tāpat jāpievieno atsauce uz autoru klasi:

```
use Illuminate\Database\Eloquent\Relations\BelongsTo;
...
public function author(): BelongsTo
{
    return $this->belongsTo(Author::class);
}
```

Tagad jebkuras grāmatas autora objektu var iegūt, izsaucot metodi `$book->author()`, un jebkura autora grāmatu masīvu var iegūt ar metodi `$author->books()`!
Protams, šo relāciju apstrādi un datu ielādi veic Laravel iebūvētās funkcijas.

URL adreses

Atveriet HTML šablonu `resources/views/layout.blade.php` un navigācijas blokā pievienojiet saiti uz jauno sadaļu:

```
<li class="nav-item">
    <a class="nav-link" href="/books">Grāmatas</a>
</li>
```

Atveriet maršrutēšanas failu `routes/web.php` un jaunajai sadaļai izveidojiet tādus pašus maršrutēšanas ierakstus, kā iepriekšējai (atcerieties arī importēt kontrolieri):

```
use App\Http\Controllers\BookController;
...

// Book routes
Route::get('/books', [BookController::class, 'list']);
Route::get('/books/create', [BookController::class, 'create']);
Route::post('/books/put', [BookController::class, 'put']);
Route::get('/books/update/{book}', [BookController::class, 'update']);
Route::post('/books/patch/{book}', [BookController::class, 'patch']);
Route::post('/books/delete/{book}', [BookController::class, 'delete']);
```

CRUD

Atveriet jaunizveidoto kontrolieri `app/Http/Controllers/BookController.php`, importējiet tajā jauno objektu modeli un nepieciešamās klases:

```
use App\Models\Author;
use App\Models\Book;
use Illuminate\View\View;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Routing\Controllers\HasMiddleware;
```

Pēc tam pievienojiet metodi, kas pārbaudīs autorizāciju:

```
class BookController extends Controller implements HasMiddleware
{
    // call auth middleware
    public static function middleware(): array
    {
        return [
            'auth',
        ];
    }
```

Izveidojiet `list()` metodi:

```php
// display all Books
public function list(): View
{
    $items = Book::orderBy('name', 'asc')->get();

    return view(
        'book.list',
        [
            'title' => 'Grāmatas',
            'items' => $items
        ]
    );
}
```

Pēc tam izveidojiet `resources/views/book/list.blade.php` skatu:

```
@extends('layout')

@section('content')

<h1>{{ $title }}</h1>

@if (count($items) > 0)

    <table class="table table-sm table-hover table-striped">
        <thead class="thead-light">
            <tr>
                <th>ID</th>
                <th>Nosaukums</th>
                <th>Autors</th>
                <th>Gads</th>
                <th>Cena</th>
                <th>Attēlot</th>
                <th> </th>
            </tr>
        </thead>
        <tbody>

        @foreach($items as $book)
            <tr>
                <td>{{ $book->id }}</td>
                <td>{{ $book->name }}</td>
                <td>{{ $book->author->name }}</td>
                <td>{{ $book->year }}</td>
                <td>&euro; {{ number_format($book->price, 2, '.') }}</td>
                <td>{!! $book->display ? '&#x2714;' : '&#x274C;' !!}</td>
                <td>
                    <a
                        href="/books/update/{{ $book->id }}"
```

```
                        class="btn btn-outline-primary btn-sm"
                    >Labot</a> /
                    <form
                        method="post"
                        action="/books/delete/{{ $book->id }}"
                        class="d-inline deletion-form"
                    >
                        @csrf
                        <button
                            type="submit"
                            class="btn btn-outline-danger btn-sm"
                        >Dzēst</button>
                    </form>
                </td>
            </tr>
        @endforeach

        </tbody>
    </table>

@else
    <p>Nav atrasts neviens ieraksts</p>
@endif

<a href="/books/create" class="btn btn-primary">Pievienot jaunu</a>

@endsection
```

(*Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Pievērsiet uzmanību sekojošiem koda fragmentiem:

- `$book->author->name` – šeit, ar modelī definēto relāciju palīdzību, tiek izmantots grāmatai piesaistītā autora vārds
- `€` – HTML simbolu kods (*entity*) eiro simbola € attēlošanai
- `number_format($book->price, 2, '.')` – izmantota PHP funkcija `number_format()`, lai grāmatu cenas attēlotu ar diviem cipariem aiz komata
- `$book->display ? '✔' : '❌'` – izmantots ternārais `if`, lai ar emoji simboliem ✔ un ✘ attēlotu grāmatas `display` statusu – arī emoji simbolus kodē ar HTML simbolu kodiem

Tagad izveidosim objektu pievienošanas funkcionalitāti. Kontrolierī pievienojiet metodi `create()`, kas sagatavos objektu pievienošanas formu:

```php
// display new Book form
public function create(): View
{
    $authors = Author::orderBy('name', 'asc')->get();

    return view(
        'book.form',
        [
            'title' => 'Pievienot grāmatu',
            'book' => new Book(),
            'authors' => $authors,
        ]
    );
}
```

(*Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Tā kā šajā lietotnē grāmatām būs jānorāda autori, tad grāmatu formai mēs nododam arī autoru masīvu `$authors`!

Kontrolierī pievienojiet arī metodi `put()`, kas veiks datu apstrādi un jauno objektu saglabāšanu:

```php
// create new Book entry
public function put(Request $request): RedirectResponse
{
    $validatedData = $request->validate([
        'name' => 'required|min:3|max:256',
        'author_id' => 'required',
        'description' => 'nullable',
        'price' => 'nullable|numeric',
        'year' => 'numeric',
        'image' => 'nullable|image',
        'display' => 'nullable',
    ]);

    $book = new Book();
    $book->name = $validatedData['name'];
    $book->author_id = $validatedData['author_id'];
    $book->description = $validatedData['description'];
    $book->price = $validatedData['price'];
    $book->year = $validatedData['year'];
    $book->display = (bool) ($validatedData['display'] ?? false);
    $book->save();

    return redirect('/books');
}
```

(*Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Šī metode jau ir ievērojami garāka par pārējām, ko līdz šim esam rakstījuši, jo nepieciešams definēt validācijas noteikumus un piešķirt vērtības visiem objekta atribūtiem.

Izveidojiet formas skatu resources/views/book/form.blade.php :

```
@extends('layout')

@section('content')

<h1>{{ $title }}</h1>

@if ($errors->any())
    <div class="alert alert-danger">Lūdzu, novērsiet radušās kļūdas!</div>
@endif

<form
    method="post"
    action="{{ $book->exists ? '/books/patch/' . $book->id : '/books/put' }}">
    @csrf

    <div class="mb-3">
        <label for="book-name" class="form-label">Nosaukums</label>

        <input
            type="text"
            id="book-name"
            name="name"
            value="{{ old('name', $book->name) }}"
            class="form-control @error('name') is-invalid @enderror"
        >

        @error('name')
            <p class="invalid-feedback">{{ $errors->first('name') }}</p>
        @enderror
    </div>

    <div class="mb-3">
        <label for="book-author" class="form-label">Autors</label>

        <select
            id="book-author"
            name="author_id"
            class="form-select @error('author_id') is-invalid @enderror"
        >
            <option value="">Norādiet autoru!</option>
                @foreach($authors as $author)
                    <option
                        value="{{ $author->id }}"
```

```
                        @if ($author->id == old('author_id', $book->author->id
?? false)) selected @endif
                    >{{ $author->name }}</option>
                @endforeach
        </select>

        @error('author_id')
            <p class="invalid-feedback">{{ $errors->first('author_id') }}</p>
        @enderror
    </div>

    <div class="mb-3">
        <label for="book-description" class="form-label">Apraksts</label>

        <textarea
            id="book-description"
            name="description"
            class="form-control @error('description') is-invalid @enderror"
        >{{ old('description', $book->description) }}</textarea>

        @error('description')
            <p class="invalid-feedback">{{ $errors->first('description') }}</p>
        @enderror
    </div>

    <div class="mb-3">
        <label for="book-year" class="form-label">Izdošanas gads</label>

        <input
            type="number" max="{{ date('Y') + 1 }}" step="1"
            id="book-year"
            name="year"
            value="{{ old('year', $book->year) }}"
            class="form-control @error('year') is-invalid @enderror"
        >

        @error('year')
            <p class="invalid-feedback">{{ $errors->first('year') }}</p>
        @enderror
    </div>

    <div class="mb-3">
        <label for="book-price" class="form-label">Cena</label>

        <input
            type="number" min="0.00" step="0.01" lang="en"
            id="book-price"
            name="price"
            value="{{ old('price', $book->price) }}"
            class="form-control @error('price') is-invalid @enderror"
        >

        @error('price')
            <p class="invalid-feedback">{{ $errors->first('price') }}</p>
        @enderror
    </div>

    // image
```

```
    <div class="mb-3">
        <div class="form-check">
            <input
                type="checkbox"
                id="book-display"
                name="display"
                value="1"
                class="form-check-input @error('display') is-invalid @enderror"
                @if (old('display', $book->display)) checked @endif
            >
            <label class="form-check-label" for="book-display">
                Publicēt ierakstu
            </label>

            @error('display')
                <p class="invalid-feedback">{{ $errors->first('display') }}</p>
            @enderror
        </div>
    </div>

    <button type="submit" class="btn btn-primary">
        {{ $book->exists ? 'Atjaunot ierakstu' : 'Pievienot ierakstu' }}
    </button>
</form>

@endsection
```

(*Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Pievienojiet dažus objektus datubāzei! Ievērojiet, ka formā ir paredzēta vieta attēlu ievadlaukam – pēc pamatfunkcionalitātes izveidošanas pievienosim arī to!

Kad datubāzē pievienoti daži ieraksti, papildiniet kontrolieri ar metodēm `update()` un `patch()`, lai ierakstus varētu rediģēt:

```php
// display Book edit form
public function update(Book $book): View
{
    $authors = Author::orderBy('name', 'asc')->get();

    return view(
        'book.form',
        [
            'title' => 'Rediģēt grāmatu',
            'book' => $book,
            'authors' => $authors,
        ]
    );
}

// update Book data
public function patch(Book $book, Request $request): RedirectResponse
{
    $validatedData = $request->validate([
        'name' => 'required|min:3|max:256',
        'author_id' => 'required',
        'description' => 'nullable',
        'price' => 'nullable|numeric',
        'year' => 'numeric',
        'image' => 'nullable|image',
        'display' => 'nullable',
    ]);

    $book->name = $validatedData['name'];
    $book->author_id = $validatedData['author_id'];
    $book->description = $validatedData['description'];
    $book->price = $validatedData['price'];
    $book->year = $validatedData['year'];
    $book->display = (bool) ($validatedData['display'] ?? false);
    $book->save();

    return redirect('/books/update/' . $book->id);
}
```

(*Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Arī šajā kontrolierī ierakstu rediģēšanas metodes ir gandrīz identiskas ierakstu izveidošanas metodēm, bet tās rediģējamo objektu saņem no Laravel.

Visbeidzot, pievienojiet arī ierakstu dzēšanas metodi:

```
// delete Book
public function delete(Book $book): RedirectResponse
{
    // dzēšam arī bildes
    $book->delete();
    return redirect('/books');
}
```

Vai pamanījāt, ka ierakstus dzēšot, arī šajā sadaļā nostrādā dialoglogs, kas lietotājam lūdz apstiprināt dzēšanu? Tas notiek tādēļ, ka arī šīs sadaļas saraksta skatā dzēšanas formām ir piešķirta klase `deletion-form`, kurai iepriekšējā nodarbībā uzrakstījām JavaScript funkciju ierakstu aizsardzībai pret nejaušu izdzēšanu!

Kad satura pievienošana, rediģēšana un dzēšana darbojas tā, kā vēlaties, varat saglabāt izmaiņas Git repozitorijā!

Attēlu augšupielāde

Kad objektu pievienošana/rediģēšana/dzēšana notestēta, pievienosim iespēju augšuplādēt attēlus.

Izveidojiet direktoriju `public/images/` !
Pēc tam atveriet Git ignorējamo failu sarakstu `.gitignore` Laravel instalācijas direktorijā, un tajā pievienojiet rindu:
`/public/images`

Tas jādara, lai augšuplādētie attēli netiktu pievienoti Git repozitorijam.

Vēl nepieciešams papildināt Laravel konfigurāciju, lai satvars augšuplādētos attēlus saglabātu publiskajā attēlu direktorijā, ko tikko izveidojāt. Atveriet failu `config/filesystems.php` un tajā pēc masīva '`public`' iekopējiet šo masīvu:

```
'public' => [
    ...
],

'uploads' => [
    'driver' => 'local',
    'root' => public_path('images'),
    'url' => env('APP_URL') . '/images',
    'visibility' => 'public',
],

's3' => [
```

(*Pelēkais teksts slīprakstā norāda failā jau esošo kodu– to nav nepieciešams kopēt!*)

Atveriet formas skatu `resources/views/book/form.blade.php` un papildiniet to! HTML formas elementam pievienojiet `enctype` atribūtu, kas atļaus ar šo formu augšuplādēt failus:

```
<form
    ...
    enctype="multipart/form-data"
>
```

(*Pelēkais teksts slīprakstā norāda failā jau esošo kodu– to nav nepieciešams kopēt!*)

Pašā formā, rindas `// image` vietā pievienojiet failu ievadlauku:

```
<div class="mb-3">
    <label for="book-image" class="form-label">Attēls</label>

    @if ($book->image)
        <img
            src="{{ asset('images/' . $book->image) }}"
            class="img-fluid img-thumbnail d-block mb-2"
            alt="{{ $book->name }}"
        >
    @endif

    <input
        type="file" accept="image/png, image/jpeg, image/webp"
        id="book-image"
        name="image"
        class="form-control @error('image') is-invalid @enderror"
    >

    @error('image')
        <p class="invalid-feedback">{{ $errors->first('image') }}</p>
    @enderror
</div>
```

(*Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Ievadlaukam `<input type="file">` ir norādīts atribūts `accept`, kurā var norādīt failu tipus, ko atļaut augšuplādēt. Šajā gadījumā norādīti attēlu tipi JPG, PNG un WEBP. Tāpat kā pārējiem ievadlaukiem, arī attēlu ievadlaukam šeit pievienots paraksts, kļūdu

apstrāde, kā arī objekta pašreizējais attēls. Pašreizējā attēla pilno URL adresi iegūst ar Laravel funkciju `asset()`.

Jāpapildina arī kontroliera `app/Http/Controllers/BookController.php` metodes `put()` un `patch()` – abās šajās metodēs pirms ieraksta saglabāšanas komandas `$book->save();` pievienojiet attēla saglabāšanas kodu:

```
$book->display = (bool) ($validatedData['display'] ?? false);

if ($request->hasFile('image')) {
    // šeit varat pievienot kodu, kas nodzēš veco bildi, ja pievieno jaunu
    $uploadedFile = $request->file('image');
    $extension = $uploadedFile->clientExtension();
    $name = uniqid();
    $book->image =  $uploadedFile->storePubliclyAs(
        '/',
        $name . '.' . $extension,
        'uploads'
    );
}

$book->save();
```

(*Pelēkais teksts slīprakstā norāda failā jau esošo kodu– to nav nepieciešams kopēt! Kopējot šo kodu, pievērsiet īpašu uzmanību, lai aizstātu visus* ***$book*** *mainīgos un* ***/books/*** *URL fragmentus ar Jūsu lietotnei atbilstošiem mainīgajiem!*)

Šajā koda fragmentā tiek veiktas šādas darbības:

1. augšuplādētā faila PHP objekts tiek saņemts no HTTP pieprasījuma;
2. mainīgajā `$extension` saglabā faila tipa paplašinājumu
3. ar PHP funkciju `uniqid()` izveido unikālu identifikatoru augšuplādētajam attēlam
4. ar Laravel metodi `storePubliclyAs()` attēlu saglabā direktorijā, kas tika norādīta `filesystems.php` failā. Attēlu saglabā nevis ar oriģinālo faila nosaukumu, bet ar tikko izveidoto identifikatoru.
5. metode `storePubliclyAs()` atgriež faila ceļu- to saglabā objektā `$book`

Grāmatu dzēšanas metodē pievienojam komandu, kas nodzēš augšuplādētos attēlus:

```php
public function delete(Book $book): RedirectResponse
{
    if ($book->image) {
        unlink(getcwd() . '/images/' . $book->image);
    }

    $book->delete();
    return redirect('/books');
}
```

Pievienojiet attēlus objektiem savā lietotnē! Kad viss darbojas, saglabājiet izmaiņas Git repozitorijā!

Ja saņemat kļūdu "`Disk [uploads] does not have a configured driver`", izpildiet komandu:

```
php artisan config:clear
```

Koda pārstrāde

Droši vien pamanījāt, ka jaunajā kontrolierī daudz koda atkārtojas – tādēļ veiksim koda pārstrādes jeb refaktoringa procesu. Pārstrādes procesā nav paredzēts mainīt lietotnes funkcionalitāti, tas tiek darīts, lai uzlabotu koda kvalitāti. Pirms refaktoringa vēlams saglabāt pašreizējo koda versiju Git!

Sāksim ar acīmredzamo- metodēs `put()` un `patch()` liela daļa koda atkārtojas, tādēļ iznesīsim to atsevišķā privātajā metodē:

```php
// validate and save Book data
private function saveBookData(Book $book, Request $request): void
{
    $validatedData = $request->validate([
        'name' => 'required|min:3|max:256',
        'author_id' => 'required',
        'description' => 'nullable',
        'price' => 'nullable|numeric',
        'year' => 'numeric',
        'image' => 'nullable|image',
        'display' => 'nullable',
    ]);

    $book->name = $validatedData['name'];
    $book->author_id = $validatedData['author_id'];
    $book->description = $validatedData['description'];
    $book->price = $validatedData['price'];
```

```
    $book->year = $validatedData['year'];
    $book->display = (bool) ($validatedData['display'] ?? false);

    if ($request->hasFile('image')) {
        // šeit varat pievienot kodu, kas nodzēš veco bildi, ja pievieno jaunu
        $uploadedFile = $request->file('image');
        $extension = $uploadedFile->clientExtension();
        $name = uniqid();
        $book->image = $uploadedFile->storePubliclyAs(
            '/',
            $name . '.' . $extension,
            'uploads'
        );
    }
    $book->save();
}

public function put(Request $request): RedirectResponse
{
    $book = new Book();
    $this->saveBookData($book, $request);
    return redirect('/books');
}

public function patch(Book $book, Request $request): RedirectResponse
{
    $this->saveBookData($book, $request);
    return redirect('/books/update/' . $book->id);
}
```

Nākamais, ko varam izdarīt- uzlabot datu pārnešanu no HTTP pieprasījuma uz objektu, ko saglabājam datubāzē. Pašlaik katra argumenta vērtību uzstādām manuāli:

```
$book->name = $validatedData['name'];
$book->author_id = $validatedData['author_id'];
$book->description = $validatedData['description'];
$book->price = $validatedData['price'];
$book->year = $validatedData['year'];
$book->display = (bool) ($validatedData['display'] ?? false);
```

Mēs varam labāk.

Atveriet modeli `app/Models/Book.php` un pievienojiet klases mainīgo `$fillable`, kurā norādītas automātiski aizpildāmās kolonnas:

```
protected $fillable = [
    'name',
    'author_id',
    'description',
    'price',
    'year',
];
```

Tagad varat nomainīt šo argumentu uzstādīšanu kontroliera metodē `saveBookData()`:

```
$book->fill($validatedData);
$book->display = (bool) ($validatedData['display'] ?? false);
```

`display` un `image` vērtību uzstādīšanu nemainām, jo šīm vērtībām veicam papildus apstrādi!

Šādu automātisku vērtību uzstādīšanu sauc par *masveida piešķiršanu* (mass assignment). Laravel satvarā to veic ar modeļu `fill()` metodi. Ņemiet vērā, ka ar `fill()` metodi var uzstādīt vērtības tikai tiem modeļa argumentiem, kas norādīti modeļa `$fillable` masīvā – tas tādēļ, lai pasargātu datu integritāti.

Vairāk par Laravel masveida piešķiršanu (un citām modeļu iespējām) lasiet dokumentācijā: https://laravel.com/docs/13.x/eloquent#mass-assignment !

Pēdējais, ko izdarīsim refaktoringa solī- izveidosim klasi grāmatu izveidošanas un rediģēšanas pieprasījumu validācijai. Liela apjoma datorsistēmās HTML formu apstrādei mēdz izmantot atsevišķas klases, kas veic validāciju, mainīgo pārvaldīšanu, reizēm arī formu HTML ģenerēšanu. Šajā lietotnē izmantosim vienkāršu pieprasījuma klasi.
Vairāk par pieprasījumu validāciju lasiet Laravel dokumentācijā:
https://laravel.com/docs/13.x/validation#form-request-validation

Ar Artisan rīku izveidojiet klasi pieprasījumu apstrādei:
`php artisan make:request BookRequest`

Jaunizveidotā klase atrodama direktorijā `app/HTTP/Requests` . Atveriet `BookRequest` klasi, tā izskatās aptuveni šādi:

```
class BookRequest extends FormRequest
{
    public function authorize(): bool
    {
        return false;
    }

    public function rules(): array
    {
        return [
            //
        ];
    }
}
```

Nomainiet metodes `authorize()` vērtību uz `true` , un metodē `rules()` iekopējiet validācijas noteikumus no `BookController` kontroliera:

```
public function authorize(): bool

{
    return true;
}

public function rules(): array
{
    return [
        'name' => 'required|min:3|max:256',
        'author_id' => 'required',
        'description' => 'nullable',
        'price' => 'nullable|numeric',
        'year' => 'numeric',
        'image' => 'nullable|image',
        'display' => 'nullable',
    ];
}
```

Pēc tam atveriet **BookController** , tajā importējiet jauno pieprasījumu klasi, un nomainiet `put()`, `patch()` un `saveBookData()` metožu argumentus, lai Laravel šajos pieprasījumos izmantotu `BookRequest` klasi:

```
use App\Http\Requests\BookRequest;


class BookController extends Controller {
    ...
    private function saveBookData(Book $book, BookRequest $request)
    ...
    public function put(BookRequest $request)
    ...
    public function patch(Book $book, BookRequest $request)
    ...
```

Pēc tam nomainiet validācijas procesu metodē `SaveBookData()` :

```
private function saveBookData(Book $book, BookRequest $request)
{
    $validatedData = $request->validated();
    $book->fill($validatedData);
```

Vēl varat pieprasījuma klasē `BookRequest` pievienot šādas metodes, kas tiks izmantotas validācijas kļūdu tulkošanai:

```
public function messages(): array
{
    return [
        'required' => 'Lauks ":attribute" ir obligāts',
        'min' => 'Laukam ":attribute" jābūt vismaz :min simbolus garam',
        'max' => 'Lauks ":attribute" nedrīkst būt garāks par :max simboliem',
        'boolean' => 'Lauka ":attribute" vērtībai jābūt "true" vai "false"',
        'unique' => 'Šāda lauka ":attribute" vērtība jau ir reģistrēta',
        'numeric' => 'Lauka ":attribute" vērtībai jābūt skaitlim',
        'image' => 'Laukā ":attribute" jāpievieno korekts attēla fails',
    ];
}
```

```php
public function attributes(): array
{
    return [
        'name' => 'nosaukums',
        'author_id' => 'autors',
        'description' => 'apraksts',
        'price' => 'cena',
        'year' => 'gads',
        'image' => 'attēls',
        'display' => 'publicēt',
    ];
}
```

Tagad grāmatu formā arī validācijas kļūdu paziņojumi būs latviski! Ideālā gadījumā gan visi lietotājiem redzamie uzraksti būtu definēti atsevišķos lokalizācijas failos- tādā veidā lietotnei viegli un ērti var pievienot citas valodas.

Pēc šo darbību veikšanas grāmatu kontrolieris izskatās ievērojami kārtīgāk- esam novērsuši koda dublikātus, samazinājuši "manuālā" darba apjomu un padarījuši grāmatu formas validāciju atkalizmantojamu.

Pārbaudiet, vai grāmatu sadaļa joprojām strādā pareizi, un tad nosūtiet izmaiņas uz Github!

Lai nostiprinātu refaktoringa iemaņas, varat refaktorēt arī autoru sadaļu, sekojot šai pamācībai.

Mājas darbs – trešā datu sadaļa

Uz nākošo nodarbību pievienojiet savai lietotnei arī trešās datu klases sadaļu- grāmatu/mūzikas/filmu žanrus, auto kategorijas u.t.t. (sk. Datu struktūra)
Šo sadaļu veidojiet vienkāršu, līdzīgu pirmajai (piemērā- Autori)!

Jums jāizveido datubāzes migrācija, modelis, kontrolieris, skatu faili – tāpat kā iepriekšējās sadaļās.
Papildiniet arī galveno datu sadaļu – datubāzes tabulu, modeļa relācijas, formas u.c. – lai šiem objektiem varētu piešķirt jaunos objektus.

Par eksistējošu datubāzes tabulu modificēšanu ar migrācijām lasiet Laravel dokumentācijā- https://laravel.com/docs/13.x/migrations#updating-tables

Pabeigto risinājumu nosūtiet uz Github!

Veiksmi!